

**LAPORAN PRAKTIKUM
STRUKTUR DATA**

“Double Linked List”

disusun Oleh:

Muhammad Fedora Argadyaksa

2511533016

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T

Asisten Praktikum:

Sherly Sukmadira



**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

2026

KATA PENGANTAR

Assalamu'alaikum Warahmatullahi Wabarakatuh

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum *Struktur Data* ini dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas praktikum pada mata kuliah Struktur Data. Dalam laporan ini, penulis memaparkan hasil praktikum yang telah dilakukan, mulai dari konsep dasar hingga implementasi struktur data seperti array, stack, queue, dan linked list dalam pemrograman.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat banyak kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari pembaca demi perbaikan di masa yang akan datang.

Tidak lupa, penulis mengucapkan terima kasih kepada dosen pengampu dan semua pihak yang telah membantu dalam pelaksanaan praktikum serta penyusunan laporan ini.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca, khususnya dalam memahami konsep dan penerapan struktur data.

Padang, 12 Mei 2025

M. Fedora Argadyaksa

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	4
1.1 Latar Belakang.....	4
1.2 Tujuan.....	4
1.3 Manfaat.....	5
BAB II PEMBAHASAN.....	6
2.1 Pengertian Double Linked List.....	6
2.2 Implementasi Stack pada Java.....	7
2.2.1 Class Node DLL (NodeDLL_2511533016).....	7
2.2.2 Class Pencarian DLL (PencarianDLL_2511533016).....	8
2.2.3 Class Insert DLL (InsertDLL_2511533016).....	10
2.2.4 Class Hapus DLL (HapusDLL_2511533016).....	12
BAB III PENUTUP	15
3.1 Kesimpulan.....	15
3.2 Saran	15
DAFTAR PUSTAKA.....	16
LAPORAN TUGAS PRAKTIKUM	17
1. Class Lagu (Lagu_2511533016)	17
2. Class Musik (Musik_2511533016)	19

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, struktur data memiliki peranan penting dalam mengatur dan mengelola data agar proses pengolahan menjadi lebih efisien. Salah satu struktur data yang sering digunakan adalah linked list. Linked list terbagi menjadi beberapa jenis, salah satunya yaitu Double Linked List. Double Linked List merupakan struktur data linear yang setiap node-nya memiliki dua buah pointer, yaitu pointer yang menunjuk ke node berikutnya (next) dan pointer yang menunjuk ke node sebelumnya (prev).

Penggunaan Double Linked List sangat membantu dalam proses manipulasi data, terutama ketika diperlukan perpindahan data secara dua arah, baik maju maupun mundur. Struktur ini lebih fleksibel dibandingkan Single Linked List karena proses penghapusan dan penyisipan node dapat dilakukan dengan lebih mudah tanpa harus selalu menelusuri node dari awal.

Materi Double Linked List dipelajari agar mahasiswa atau programmer dapat memahami konsep hubungan antar node secara dua arah serta mampu mengimplementasikan operasi-operasi dasar seperti insert, delete, traversal, dan searching. Pemahaman terhadap struktur data ini sangat penting karena banyak diterapkan dalam berbagai sistem komputer, seperti fitur undo-redo pada aplikasi, navigasi browser, playlist musik, dan sistem manajemen data lainnya.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum pekan 6 tentang Double Linked List adalah sebagai berikut:

1. Memahami konsep dasar struktur data Double Linked List
2. Mengetahui perbedaan antara Single Linked List dan Double Linked List.
3. Mampu mengimplementasikan Single Linked List dalam pemrograman.

1.3 Manfaat

Manfaat yang diharapkan dari pelaksanaan praktikum pekan 6 tentang Double Linked List adalah sebagai berikut:

1. Mempermudah pengelolaan data yang membutuhkan akses dua arah.
2. Membantu proses penyisipan dan penghapusan data menjadi lebih efisien.
3. Melatih kemampuan analisis dan pemecahan masalah dalam pembuatan program

BAB II

PEMBAHASAN

2.1 Pengertian Double Linked List

Double Linked List adalah struktur data dalam pemrograman yang digunakan untuk menyimpan sekumpulan data yang saling terhubung. Pada struktur ini, setiap data disimpan di dalam sebuah node. Setiap node memiliki tiga bagian, yaitu data, penunjuk ke data berikutnya (next), dan penunjuk ke data sebelumnya (prev).

Sederhananya, Double Linked List bisa diibaratkan seperti rangkaian gerbong kereta yang saling terhubung dua arah. Jadi, kita bisa bergerak maju ke gerbong berikutnya atau kembali ke gerbong sebelumnya dengan mudah. Karena itulah Double Linked List disebut sebagai linked list dua arah.

Berbeda dengan Single Linked List yang hanya bisa berjalan satu arah, Double Linked List lebih fleksibel karena data dapat ditelusuri dari depan ke belakang maupun dari belakang ke depan. Hal ini membuat proses penambahan, penghapusan, dan pencarian data menjadi lebih mudah dilakukan.

Biasanya, node pertama disebut head dan node terakhir disebut tail. Pada node pertama tidak ada node sebelumnya sehingga bagian prev bernilai kosong (null), sedangkan pada node terakhir tidak ada node sesudahnya sehingga bagian next juga bernilai null.

Double Linked List sering digunakan dalam berbagai program atau aplikasi, misalnya pada fitur tombol back dan forward di browser, playlist musik, fitur undo dan redo pada aplikasi editor, serta sistem navigasi data lainnya. Dengan mempelajari Double Linked List, seseorang dapat memahami bagaimana data saling terhubung dan bagaimana cara mengelola data secara lebih efisien dalam pemrograman.

2.2 Implementasi Stack pada Java

Pada praktikum pekan 6 mahasiswa disuruh untuk mengikuti intruksi dari dosen. Yaitu menuliskan kembali kode program yang sudah ada di layar TV. Kode program yang dibuat tentang Double Linked List, di bawah ini kode program yang dituliskan pada praktikum pekan 6.

2.2.1 Class Node DLL (NodeDLL_2511533016)

```

1 package Pekan6_2511533016;
2
3 public class NodeDLL_2511533016 {
4     // mendefinisikan kelas node
5     int data_3016; // data
6     NodeDLL_2511533016 next_3016;
7     NodeDLL_2511533016 prev_3016;
8
9     public NodeDLL_2511533016(int data_3016) {
10         this.data_3016 = data_3016;
11         this.prev_3016 = null;
12         this.next_3016 = null;
13     }
14 }
15

```

Gambar 2.2.1

Kode di atas merupakan definisi kelas Node untuk struktur data Doubly Linked List yang ditulis menggunakan bahasa pemrograman Java. Program ini berada dalam package Pekan6_2511533016 yang menunjukkan bahwa kode ini dibuat dalam konteks tugas perkuliahan pada minggu keenam. Kelas utama yang didefinisikan bernama NodeDLL_2511533016 dimana penamaan tersebut mengikuti konvensi yang sudah ditentukan.

Di dalam kelas tersebut terdapat tiga atribut utama yang menjadi inti dari sebuah node. Atribut data_3016 bertipe integer berfungsi untuk menyimpan nilai atau data yang dibawa oleh node. Kemudian ada next_3016 dan prev_3016 yang keduanya bertipe NodeDLL_2511533016, berfungsi sebagai pointer atau penunjuk ke node berikutnya dan node sebelumnya. Inilah yang membedakan Doubly Linked List dari Singly Linked List, karena setiap node memiliki dua arah koneksi bukan hanya satu.

Bagian terakhir adalah constructor NodeDLL_2511533016(int data_3016) yang akan dieksekusi secara otomatis ketika sebuah node baru dibuat. Di dalamnya, nilai data langsung diinisialisasi sesuai parameter yang diberikan,

sementara kedua pointer yaitu prev_3016 dan next_3016 diset bernilai null. Hal ini wajar karena node yang baru saja dibuat belum memiliki hubungan dengan node manapun, dan koneksi tersebut baru akan terbentuk ketika node dimasukkan ke dalam struktur linked list secara keseluruhan.

Kesimpulan: Kode program ini merupakan implementasi struktur data **Doubly Linked List (DLL)** dalam bahasa Java, yang mendefinisikan sebuah Node sebagai komponen dasar pembentuk linked list dua arah. Setiap node mampu menyimpan data sekaligus terhubung ke node sebelum dan sesudahnya.

2.2.2 Class Pencarian DLL (PencarianDLL_2511533016)

```

1 package Pekan6_2511533016;
2
3 public class PenelusuranDll_2511533016 {
4     // fungsi penelusuran maju
5     static void forwardTraversal_2511533016(NodeDLL_2511533016 head_3016) {
6         // memulai penelusuran dari head
7         NodeDLL_2511533016 curr_3016 = head_3016;
8         // lanjutkan sampai akhir
9         while (curr_3016 != null) {
10             // print data
11             System.out.print(curr_3016.data_3016 + " <-> ");
12             // pindah ke node berikutnya
13             curr_3016 = curr_3016.next_3016;
14         }
15         // print akhir
16         System.out.println();
17     }
18     // fungsi penelusuran mundur
19     static void backwardTraversal_2511533016(NodeDLL_2511533016 tail_3016) {
20         // mulai dari akhir
21         NodeDLL_2511533016 curr_3016 = tail_3016;
22         // lanjut sampai head
23         while (curr_3016 != null) {
24             // cetak data
25             System.out.print(curr_3016.data_3016 + " <-> ");
26             // pindah ke node sebelumnya
27             curr_3016 = curr_3016.prev_3016;
28         }
29         // cetak akhir
30         System.out.println();
31     }
32     public static void main(String[] args) {
33         // cetak DLL
34         NodeDLL_2511533016 head_3016 = new NodeDLL_2511533016(1);
35         NodeDLL_2511533016 second_3016 = new NodeDLL_2511533016(2);
36         NodeDLL_2511533016 third_3016 = new NodeDLL_2511533016(3);
37
38         head_3016.next_3016 = second_3016;
39         second_3016.prev_3016 = head_3016;
40         second_3016.next_3016 = third_3016;
41         third_3016.prev_3016 = second_3016;
42
43         System.out.println("Penelusuran maju : ");
44         forwardTraversal_2511533016(head_3016);
45
46         System.out.println("penelusuran mundur : ");
47         backwardTraversal_2511533016(third_3016);
48     }
49 }
50

```

Penelusuran maju :
 1 <-> 2 <-> 3 <->
 penelusuran mundur :
 3 <-> 2 <-> 1 <->

Gambar 2.2.2

Kode di atas adalah implementasi penelusuran Doubly Linked List yang ditulis dalam bahasa Java dan masih berada dalam package Pekan6_2511533016. Kelas utamanya bernama PenelusuranDLL_2511533016 yang secara keseluruhan

berisi dua fungsi penelusuran dan satu fungsi utama untuk menjalankan program.

Fungsi pertama adalah `forwardTraversal_2511533016` yang menerima parameter `head_3016` sebagai titik awal penelusuran. Di dalamnya, variabel `curr_3016` diinisialisasi dari node head, kemudian program akan terus berjalan melewati setiap node menggunakan perulangan `while` selama nilai `curr_3016` tidak sama dengan `null`. Setiap kali perulangan berjalan, data dari node yang sedang dikunjungi akan dicetak, lalu pointer berpindah ke node berikutnya melalui `curr_3016.next_3016`. Ini adalah penelusuran dari depan ke belakang.

Fungsi kedua adalah `backwardTraversalD_2511533016` yang cara kerjanya hampir sama dengan fungsi pertama, hanya saja penelusuran dimulai dari node tail atau node paling akhir. Variabel `curr_3016` diinisialisasi dari `tail_3016`, lalu setiap iterasi berpindah ke node sebelumnya menggunakan `curr_3016.prev_3016`. Hasilnya adalah data yang tercetak dalam urutan terbalik dari belakang ke depan.

Pada fungsi main, program membangun linked list sederhana dengan tiga node yang masing-masing berisi nilai 1, 2, dan 3. Ketiga node tersebut kemudian dihubungkan secara manual dengan mengatur atribut `next` dan `prev` pada masing-masing node, sehingga membentuk rantai `1 <-> 2 <-> 3`. Setelah linked list terbentuk, program memanggil kedua fungsi traversal tadi untuk menampilkan hasilnya. Penelusuran maju dimulai dari `head_3016` yang bernilai 1, sedangkan penelusuran mundur dimulai dari `third_3016` yang bernilai 3.

Kesimpulan: Kode program ini merupakan implementasi **penelusuran (traversal) pada struktur data Doubly Linked List (DLL)** dalam bahasa Java, yang mendemonstrasikan dua arah penelusuran yaitu maju (`forward`) dari head ke tail, dan mundur (`backward`) dari tail ke head menggunakan pointer `next` dan `prev` pada setiap node.

2.2.3 Class Insert DLL (InsertDLL_2511533016)

```

1 package Pekan6_2511533016;
2
3 public class InsertDLL_2511533016 {
4     // Memasukkan node di awal DLL
5     static NodeDLL_2511533016 insertBegin_2511533016(NodeDLL_2511533016 head_3016, int data_3016) {
6         // buat node baru
7         NodeDLL_2511533016 new_node_3016 = new NodeDLL_2511533016(data_3016);
8         // jadikan pointer pertama head
9         new_node_3016.next_3016 = head_3016;
10        // jadikan pointer terakhir head ke new_node
11        if (head_3016 != null) {
12            head_3016.prev_3016 = new_node_3016;
13        }
14        return new_node_3016;
15    }
16    // fungsi menambahkan node di akhir
17    static NodeDLL_2511533016 insertEnd_2511533016(NodeDLL_2511533016 head_3016, int new_data_3016) {
18        // buat node baru
19        NodeDLL_2511533016 new_node_3016 = new NodeDLL_2511533016(new_data_3016);
20        // jika dll null jadikan head
21        if (head_3016 == null) {
22            head_3016 = new_node_3016;
23        }
24        else {
25            NodeDLL_2511533016 curr_3016 = head_3016;
26            while (curr_3016.next_3016 != null) {
27                curr_3016 = curr_3016.next_3016;
28            }
29            curr_3016.next_3016 = new_node_3016;
30            new_node_3016.prev_3016 = curr_3016;
31        }
32        return head_3016;
33    }
34    // fungsi memasukkan node di posisi tertentu
35    static NodeDLL_2511533016 insertAtPosition_2511533016(NodeDLL_2511533016 head_3016, int pos_3016, int new_data_3016) {
36        // buat node baru
37        NodeDLL_2511533016 new_node_3016 = new NodeDLL_2511533016(new_data_3016);
38        if (pos_3016 == 1) {
39            new_node_3016.next_3016 = head_3016;
40            if (head_3016 != null) {
41                head_3016.prev_3016 = new_node_3016;
42            }
43            head_3016 = new_node_3016;
44            return head_3016;
45        }
46        NodeDLL_2511533016 curr_3016 = head_3016;
47        for (int i = 1; i < pos_3016 - 1 && curr_3016 != null; ++i) {
48            curr_3016 = curr_3016.next_3016;
49        }
50        if (curr_3016 == null) {
51            System.out.println("Posisi tidak ada");
52            return head_3016;
53        }
54        new_node_3016.next_3016 = curr_3016;
55        new_node_3016.prev_3016 = curr_3016;
56        curr_3016.next_3016 = new_node_3016;
57        if (new_node_3016.next_3016 != null) {
58            new_node_3016.next_3016.prev_3016 = new_node_3016;
59        }
60        return head_3016;
61    }
62    public static void printList_2511533016(NodeDLL_2511533016 head_3016) {
63        NodeDLL_2511533016 curr_3016 = head_3016;
64        while (curr_3016 != null) {
65            System.out.print(curr_3016.data_3016 + " <-> ");
66            curr_3016 = curr_3016.next_3016;
67        }
68        System.out.println();
69    }
70    public static void main(String[] args) {
71        // membuat dll 2 <-> 3 <-> 5
72        NodeDLL_2511533016 head_3016 = new NodeDLL_2511533016(2);
73        head_3016.next_3016 = new NodeDLL_2511533016(3);
74        head_3016.next_3016.prev_3016 = head_3016;
75        head_3016.next_3016.next_3016 = new NodeDLL_2511533016(5);
76        head_3016.next_3016.prev_3016 = head_3016;
77        // cetak DLL awal
78        System.out.print("DLL Awal: ");
79        printList_2511533016(head_3016);
80        // tambah 1 di awal
81        head_3016 = insertBegin_2511533016(head_3016, 1);
82        System.out.print("simpul 1 ditambah di awal: ");
83        printList_2511533016(head_3016);
84        // tambah 6 di akhir
85        System.out.print("simpul 6 ditambah di akhir: ");
86        int data_3016 = 6;
87        head_3016 = insertEnd_2511533016(head_3016, data_3016);
88        printList_2511533016(head_3016);
89        // memasukkan node ke di posisi 4
90        System.out.print("tambah node 4 di posisi 4: ");
91        int data2_3016 = 4;
92        int pos_3016 = 4;
93        head_3016 = insertAtPosition_2511533016(head_3016, pos_3016, data2_3016);
94        printList_2511533016(head_3016);
95    }
96 }

```

DLL Awal: 2 <-> 3 <-> 5 <->
 simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
 simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
 tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->

Gambar 2.2.3

Kode di atas adalah implementasi operasi insert pada Doubly Linked List yang masih berada dalam package Pekan6_2511533016. Kelas utamanya bernama insertDLL_2511533016 yang berisi tiga fungsi insert, satu fungsi untuk menampilkan list, dan satu fungsi utama untuk menjalankan program secara

keseluruhan.

Fungsi pertama adalah `insertBegin_2511533016` yang bertugas menambahkan node baru di posisi paling awal atau depan dari linked list. Cara kerjanya adalah dengan membuat node baru terlebih dahulu, kemudian mengarahkan pointer `next` dari node baru tersebut ke node head yang lama. Jika head sebelumnya tidak kosong atau null, maka pointer `prev` dari head lama akan diarahkan balik ke node baru. Setelah semua koneksi terbentuk, node baru tersebut dikembalikan sebagai head yang baru.

Fungsi kedua adalah `insertEnd_2511533016` yang berfungsi menambahkan node baru di posisi paling akhir atau belakang linked list. Jika linked list masih kosong, maka node baru langsung dijadikan head. Namun jika linked list sudah ada isinya, program akan menelusuri seluruh node menggunakan perulangan `while` hingga menemukan node paling akhir, lalu menghubungkan node tersebut dengan node baru melalui pointer `next` dan `prev`.

Fungsi ketiga adalah `insertAtPosition_2511533016` yang lebih fleksibel karena bisa menyisipkan node di posisi manapun sesuai indeks yang diberikan. Jika posisi yang diminta adalah 1, maka prosesnya sama seperti insert di awal. Jika bukan, program akan menelusuri node satu per satu hingga mencapai posisi yang dituju. Apabila posisi yang diminta melebihi panjang list yang ada, program akan mencetak pesan "Posisi tidak ada" dan mengembalikan head tanpa perubahan. Jika posisi valid, node baru akan disisipkan dengan mengatur ulang pointer `prev` dan `next` dari node-node di sekitarnya agar koneksi tetap terjaga di kedua arah.

Fungsi `printList_2511533016` bertugas mencetak semua isi linked list dari head hingga node terakhir dengan format yang dipisahkan tanda `<->`. Pada fungsi `main`, program membangun linked list awal dengan nilai 2 `<->` 3 `<->` 5, kemudian secara berurutan menguji ketiga fungsi insert. Pertama menambahkan nilai 1 di awal sehingga menjadi 1 `<->` 2 `<->` 3 `<->` 5, lalu menambahkan nilai 6 di akhir menjadi 1 `<->` 2 `<->` 3 `<->` 5 `<->` 6, dan terakhir menyisipkan nilai 4 di posisi ke-4 sehingga susunan akhirnya menjadi 1 `<->` 2 `<->` 3 `<->` 4 `<->` 5 `<->` 6.

Kesimpulan: Kode program ini merupakan implementasi **operasi penyisipan**

node (insert) pada struktur data Doubly Linked List (DLL) dalam bahasa Java, yang mendemonstrasikan tiga cara penyisipan node yaitu di awal list, di akhir list, dan di posisi tertentu sesuai indeks yang diinginkan.

2.2.4 Class Hapus DLL (HapusDLL_2511533016)

```

1 package Pekan6_2511533016;
2
3 public class HapusDLL_2511533016 {
4     // Fungsi menghapus node awal
5     public static NodeDll_2511533016 delHead_2511533016(NodeDll_2511533016 head_3016) {
6         if (head_3016 == null) {
7             return null;
8         }
9         NodeDll_2511533016 temp_3016 = head_3016;
10        head_3016 = head_3016.next_3016;
11        if (head_3016 != null) {
12            head_3016.prev_3016 = null;
13        }
14        return head_3016;
15    }
16
17    // Fungsi di akhir
18    public static NodeDll_2511533016 delLast_2511533016(NodeDll_2511533016 head_3016) {
19        if (head_3016 == null) {
20            return null;
21        }
22        if (head_3016.next_3016 == null) {
23            return null;
24        }
25        NodeDll_2511533016 curr_3016 = head_3016;
26        while (curr_3016.next_3016 != null) {
27            curr_3016 = curr_3016.next_3016;
28        }
29        // update pointer previous node
30        if (curr_3016.prev_3016 != null) {
31            curr_3016.prev_3016.next_3016 = null;
32        }
33        return head_3016;
34    }
35
36    // Fungsi menghapus node posisi tertentu
37    public static NodeDll_2511533016 delPos_2511533016(NodeDll_2511533016 head_3016, int pos_3016) {
38        if (head_3016 == null) {
39            return head_3016;
40        }
41        NodeDll_2511533016 curr_3016 = head_3016;
42        // temp node yang akan dihapus
43        for (int i = 1; curr_3016 != null && i < pos_3016; ++i) {
44            curr_3016 = curr_3016.next_3016;
45        }
46        // jika posisi tidak ditemukan
47        if (curr_3016 == null) {
48            return head_3016;
49        }
50        // update pointer
51        if (curr_3016.prev_3016 != null) {
52            curr_3016.prev_3016.next_3016 = curr_3016.next_3016;
53        }
54        if (curr_3016.next_3016 != null) {
55            curr_3016.next_3016.prev_3016 = curr_3016.prev_3016;
56        }
57        // jika yang dihapus head
58        if (head_3016 == curr_3016) {
59            head_3016 = curr_3016.next_3016;
60        }
61        return head_3016;
62    }
63
64    // Fungsi mencetak DLL
65    public static void printList_2511533016(NodeDll_2511533016 head_3016) {
66        NodeDll_2511533016 curr_3016 = head_3016;
67        while (curr_3016 != null) {
68            System.out.print(curr_3016.data_3016 + " ");
69            curr_3016 = curr_3016.next_3016;
70        }
71        System.out.println();
72    }
73
74    public static void main(String[] args) {
75        // buat sebuah DLL
76        NodeDll_2511533016 head_3016 = new NodeDll_2511533016(1);
77        head_3016.next_3016 = new NodeDll_2511533016(2);
78        head_3016.next_3016.prev_3016 = head_3016;
79        head_3016.next_3016.next_3016 = new NodeDll_2511533016(3);
80        head_3016.next_3016.next_3016.prev_3016 = head_3016.next_3016;
81        head_3016.next_3016.next_3016.next_3016 = new NodeDll_2511533016(4);
82        head_3016.next_3016.next_3016.next_3016.prev_3016 = head_3016.next_3016.next_3016;
83        head_3016.next_3016.next_3016.next_3016.next_3016 = new NodeDll_2511533016(5);
84        head_3016.next_3016.next_3016.next_3016.next_3016.prev_3016 = head_3016.next_3016.next_3016.next_3016;
85
86        System.out.print("DLL Awal : ");
87        printList_2511533016(head_3016);
88
89        System.out.print("Setelah head dihapus : ");
90        head_3016 = delHead_2511533016(head_3016);
91        printList_2511533016(head_3016);
92
93        System.out.print("Setelah node terakhir dihapus : ");
94        head_3016 = delLast_2511533016(head_3016);
95        printList_2511533016(head_3016);
96
97        System.out.print("Menghapus node ke 2 : ");
98        head_3016 = delPos_2511533016(head_3016, 2);
99        printList_2511533016(head_3016);
100    }
101 }

```

Gambar 2.2.4

Kode di atas adalah implementasi operasi hapus pada Doubly Linked List yang masih berada dalam package Pekan6_2511533016. Kelas utamanya bernama hapusDLL_2511533016 yang berisi tiga fungsi delete, satu fungsi untuk

menampilkan isi list, dan satu fungsi utama untuk menjalankan dan menguji semua operasi penghapusan tersebut..

Fungsi pertama adalah `delHead_2511533016` yang bertugas menghapus node yang berada di posisi paling depan atau head dari linked list. Pertama program mengecek apakah list kosong, jika iya langsung dikembalikan null. Jika tidak kosong, program menyimpan referensi head sementara ke variabel `temp_3016`, lalu memindahkan head ke node berikutnya. Setelah itu pointer `prev` dari head yang baru diset ke null agar tidak lagi menunjuk ke node yang sudah dihapus, kemudian head baru tersebut dikembalikan sebagai hasil.

Fungsi kedua adalah `delLast_2511533016` yang menghapus node di posisi paling akhir dari linked list. Jika list kosong maka langsung dikembalikan null, dan jika list hanya punya satu node maka node tersebut langsung dihapus dengan mengembalikan null. Jika lebih dari satu node, program menelusuri seluruh list menggunakan perulangan `while` hingga mencapai node paling akhir. Setelah ditemukan, pointer `next` dari node sebelumnya diset ke null sehingga node terakhir tersebut terputus dari rantai linked list.

Fungsi ketiga adalah `delPos_2511533016` yang paling fleksibel karena bisa menghapus node di posisi manapun berdasarkan indeks yang diberikan. Program akan menelusuri node satu per satu menggunakan perulangan `for` hingga mencapai posisi yang dituju. Jika posisi tidak ditemukan atau melebihi panjang list, program langsung mengembalikan head tanpa perubahan. Jika posisi valid, program mengatur ulang pointer `next` dari node sebelumnya dan pointer `prev` dari node sesudahnya agar keduanya saling terhubung langsung, seolah node yang dihapus tidak pernah ada. Jika node yang dihapus ternyata adalah head, maka head diperbarui ke node berikutnya.

Fungsi `printList_2511533016` bertugas mencetak seluruh isi linked list dari head hingga akhir dengan setiap data dipisahkan spasi. Pada fungsi `main`, program membangun linked list awal dengan lima node bernilai 1 <-> 2 <-> 3 <-> 4 <-> 5. Kemudian program menguji ketiga fungsi delete secara berurutan, pertama menghapus head sehingga list menjadi 2 <-> 3 <-> 4 <-> 5, lalu menghapus node terakhir menjadi 2 <-> 3 <-> 4, dan terakhir menghapus node di posisi ke-2 sehingga susunan akhir list menjadi 2 <-> 4.

Kesimpulan: Kode program ini merupakan implementasi **operasi penghapusan node (delete) pada struktur data Doubly Linked List (DLL)** dalam bahasa Java, yang mendemonstrasikan tiga cara penghapusan node yaitu menghapus node di awal list, di akhir list, dan di posisi tertentu sesuai indeks yang diinginkan.

BAB III

PENUTUP

3.1 Kesimpulan

Double Linked List (DLL) adalah struktur data linked list yang setiap nodenya memiliki dua pointer yaitu next untuk menunjuk node berikutnya dan prev untuk menunjuk node sebelumnya. Berbeda dengan Single Linked List, DLL memungkinkan penelusuran data dari dua arah sekaligus, baik maju maupun mundur.

Dari keempat program yang telah dibuat, semuanya saling melengkapi dalam membangun pemahaman DLL secara utuh. NodeDLL menjadi fondasi dengan mendefinisikan struktur dasar sebuah node. PenelusuranDLL menunjukkan cara membaca data menggunakan pointer next dan prev dari dua arah berbeda. InsertDLL memperlihatkan cara menambahkan node baru di awal, akhir, maupun posisi tertentu dengan memperbarui pointer di sekitarnya. Sedangkan HapusDLL melengkapi operasi dasar DLL dengan menghapus node dari posisi manapun tanpa memutus rantai koneksi antar node yang tersisa.

Secara keseluruhan, Double Linked List adalah struktur data yang fleksibel dan efisien untuk berbagai kebutuhan seperti browser history, fitur undo-redo, maupun navigasi playlist. Meskipun membutuhkan memori lebih karena pointer tambahan, kemampuan dua arahnya menjadi keunggulan utama yang membuat DLL lebih unggul dibanding Single Linked List dalam banyak situasi.

3.2 Saran

Adapun saran yang dapat diberikan berdasarkan materi ini adalah sebagai berikut:

1. Pastikan setiap perubahan pada linked list (insert/delete) dilakukan dengan benar agar tidak terjadi kehilangan data atau error (misalnya null pointer).
2. Sebaiknya tambahkan pengecekan posisi atau kondisi list agar program lebih aman, seperti validasi posisi saat insert atau delete.
3. Pisahkan fungsi-fungsi ke dalam class yang berbeda (misalnya class LinkedList dan Node) agar struktur program lebih terorganisir.

DAFTAR PUSTAKA

- [1] T. H. Cormen, C. E. Leiserson dan R. L. Rivest, “Introducing to Algorithms,” 2009. [Online].
- [2] M. T. Goodrich, R. Tamassia dan M. H. Goldwasser, “Data Structures and Algorithms in Java,” 2014. [Online].

LAPORAN TUGAS PRAKTIKUM

SOAL:

Buatlah program sederhana untuk mengelola playlist lagu menggunakan Double Linked List. Setiap lagu adalah node yang memiliki pointer ke lagu sebelumnya (prev) dan lagu berikutnya (next). Program harus memungkinkan pengguna menambah lagu, menghapus lagu, dan melihat daftar lagu dari dua arah.

1. Class Lagu (Lagu_2511533016)

```
1. package Pekan6_2511533016;
2.
3. public class Lagu_2511533016 {
4.     String judul_3016;
5.     String penyanyi_3016;
6.     Lagu_2511533016 next_3016;
7.     Lagu_2511533016 prev_3016;
8.
9.     // Constructor
10.    public Lagu_2511533016(String judul_3016, String
    penyanyi_3016) {
11.        this.judul_3016 = judul_3016;
12.        this.penyanyi_3016 = penyanyi_3016;
13.        this.next_3016 = null;
14.        this.prev_3016 = null;
15.    }
16.
17.    // Getter
18.    public String getJudul_3016() {
19.        return judul_3016;
20.    }
21.    public String getPenyanyi_3016() {
22.        return penyanyi_3016;
23.    }
24.
25.    // Setter
26.    public void setJudul_3016 (String judul_3016) {
27.        this.judul_3016 = judul_3016;
28.    }
29.    public void setPenyanyi_3016 (String penyanyi_3016) {
30.        this.penyanyi_3016 = penyanyi_3016;
31.    }
32. }
```

Gambar 1

Kode di atas adalah definisi kelas Lagu_2511533016 yang masih berada dalam package Pekan6_2511533016. Berbeda dengan node sebelumnya yang hanya menyimpan data bertipe integer, kelas ini merepresentasikan sebuah node yang menyimpan informasi lebih kompleks yaitu data lagu, sehingga lebih mencerminkan penggunaan Double Linked List dalam kasus nyata.

Di dalam kelas terdapat empat atribut utama. Dua atribut pertama yaitu judul_3016 dan penyanyi_3016 bertipe String yang berfungsi menyimpan informasi dari sebuah lagu. Dua atribut berikutnya yaitu next_3016 dan prev_3016 bertipe Lagu_2511533016 yang berfungsi sebagai pointer penghubung ke node lagu berikutnya dan sebelumnya, sama seperti konsep DLL pada umumnya.

Bagian constructor Lagu_2511533016 menerima dua parameter yaitu judul dan penyanyi, kemudian langsung menginisialisasi kedua atribut tersebut saat node baru dibuat. Sementara pointer next dan prev diset null karena node yang baru dibuat belum terhubung ke node manapun.

Yang membedakan kelas ini dari node sebelumnya adalah adanya penerapan konsep OOP melalui getter dan setter. Fungsi getter yaitu getJudul_3016 dan getPenyanyi_3016 digunakan untuk mengambil nilai atribut dari luar kelas, sedangkan fungsi setter yaitu setJudul_3016 dan setPenyanyi_3016 digunakan untuk mengubah nilai atribut tersebut. Pendekatan ini membuat data di dalam node lebih terlindungi dan pengelolaannya menjadi lebih terstruktur sesuai prinsip enkapsulasi dalam OOP.

Kesimpulan: Kode program ini merupakan implementasi **Node berbasis objek untuk struktur data Doubly Linked List (DLL)** dalam bahasa Java, yang merepresentasikan sebuah lagu sebagai node dengan menerapkan konsep Object Oriented Programming (OOP) melalui penggunaan constructor, getter, dan setter.

2. Class Musik (Musik_2511533016)

```

1. package Pekan6_2511533016;
2. import java.util.Scanner;
3.
4. public class Musik_2511533016 {
5.     private Lagu_2511533016 head_3016;
6.     private Lagu_2511533016 tail_3016;
7.
8.     // 1. Tambah Lagu di Akhir
9.     public void tambahLagu_3016(String judul_3011, String
penyanyi_3011) {
10.         Lagu_2511533016 baru_3016 = new
Lagu_2511533016(judul_3011, penyanyi_3011);
11.         if (head_3016 == null) {
12.             head_3016 = tail_3016 = baru_3016;
13.         } else {
14.             tail_3016.next_3016 = baru_3016;
15.             baru_3016.prev_3016 = tail_3016;
16.             tail_3016 = baru_3016;
17.         }
18.         System.out.println("Lagu berhasil ditambahkan!");
19.     }
20.
21.     // 2. Hapus Lagu Awal
22.     public void hapusLaguAwal_3016() {
23.         if (head_3016 == null) {
24.             System.out.println("Playlist kosong!");
25.         } else {
26.             head_3016 = head_3016.next_3016;
27.             if (head_3016 != null) head_3016.prev_3016 = null;
28.             else tail_3016 = null;
29.             System.out.println("Lagu pertama berhasil dihapus.");
30.         }
31.     }
32.
33.     // 3. Tampil Maju
34.     public void tampilMaju_3016() {
35.         if (head_3016 == null) System.out.println("Playlist
kosong.");
36.         Lagu_2511533016 curr = head_3016;
37.         while (curr != null) {
38.             System.out.println(curr.judul_3016 + " - " +
curr.penyanyi_3016);
39.             curr = curr.next_3016;
40.         }
41.     }
42.
43.     // 4. Tampil Mundur
44.     public void tampilMundur_3016() {
45.         if (tail_3016 == null) System.out.println("Playlist
kosong.");
46.         Lagu_2511533016 curr = tail_3016;
47.         while (curr != null) {
48.             System.out.println(curr.judul_3016 + " - " +
curr.penyanyi_3016);

```

```

49.         curr = curr.prev_3016;
50.     }
51. }
52.
53. // 5. Cari Lagu
54. public void carilagu_3016(String judul_3011) {
55.     Lagu_2511533016 curr = head_3016;
56.     boolean found = false;
57.     while (curr != null) {
58.         if (curr.judul_3016.equalsIgnoreCase(judul_3011)) {
59.             System.out.println("Lagu ditemukan: " +
curr.judul_3016 + " oleh " + curr.penyanyi_3016);
60.             found = true;
61.         }
62.         curr = curr.next_3016;
63.     }
64.     if (!found) System.out.println("Lagu tidak ditemukan.");
65. }
66.
67. public static void main(String[] args) {
68.     Musik_2511533016 playlist = new Musik_2511533016();
69.     Scanner sc = new Scanner(System.in);
70.     int pilihan = 0;
71.
72.     while (pilihan != 6) {
73.         System.out.println("\n=== Playlist Musik NIM:
2511533016 ===");
74.         System.out.println("1. Tambah Lagu");
75.         System.out.println("2. Hapus Lagu Pertama");
76.         System.out.println("3. Lihat Playlist (Maju)");
77.         System.out.println("4. Lihat Playlist (Mundur)");
78.         System.out.println("5. Cari Lagu");
79.         System.out.println("6. Keluar");
80.         System.out.print("Pilihan: ");
81.         pilihan = sc.nextInt();
82.         sc.nextLine();
83.
84.         switch (pilihan) {
85.             case 1:
86.                 System.out.print("Judul: ");
87.                 String j = sc.nextLine();
88.                 System.out.print("Penyanyi: ");
89.                 String p = sc.nextLine();
90.                 playlist.tambahLagu_3016(j, p);
91.                 break;
92.             case 2:
93.                 playlist.hapusLaguAwal_3016();
94.                 break;
95.             case 3:
96.                 playlist.tampilMaju_3016();
97.                 break;
98.             case 4:
99.                 playlist.tampilMundur_3016();
100.                break;
101.             case 5:
102.                 System.out.print("Masukkan judul lagu:
");

```

```

103.         String cari = sc.nextLine();
104.         playlist.cariLagu_3016(cari);
105.         break;
106.     case 6:
107.         System.out.println("Program selesai.");
108.         break;
109.     default:
110.         System.out.println("Pilihan tidak
    valid!");
111.     }
112. }
113. sc.close();
114. }
115. }

```

Gambar 2

Kode di atas adalah implementasi kelas Musik_2511533016 yang masih berada dalam package Pekan6_2511533016. Kelas ini berperan sebagai pengelola playlist musik yang menggunakan struktur Doubly Linked List dengan memanfaatkan kelas Lagu_2511533016 sebagai nodenya. Di dalam kelas ini terdapat dua atribut utama yaitu head_3016 dan tail_3016 yang masing-masing menunjuk ke lagu pertama dan lagu terakhir dalam playlist.

Fungsi pertama adalah tambahLagu_3016 yang bertugas menambahkan lagu baru di posisi paling akhir playlist. Jika playlist masih kosong, lagu baru langsung dijadikan head sekaligus tail. Jika sudah ada isinya, lagu baru dihubungkan ke node terakhir melalui pointer next dan prev, kemudian tail diperbarui ke lagu baru tersebut.

Fungsi kedua adalah hapusLaguAwal_3016 yang menghapus lagu yang berada di posisi paling depan playlist. Jika playlist kosong program akan mencetak pesan peringatan, jika tidak maka head dipindahkan ke node berikutnya dan pointer prev dari head baru diset ke null. Jika setelah penghapusan playlist menjadi kosong, tail juga ikut diset null.

Fungsi ketiga dan keempat adalah tampilMaju_3016 dan tampilMundur_3016 yang menampilkan seluruh isi playlist dalam dua arah berbeda. Tampil maju menelusuri dari head ke tail menggunakan pointer next, sedangkan tampil mundur menelusuri dari tail ke head menggunakan pointer prev, menampilkan judul dan penyanyi dari setiap lagu yang ditemukan.

Fungsi kelima adalah cariLagu_3016 yang menerima input judul lagu dan

menelusuri seluruh playlist untuk mencari kecocokan menggunakan metode equalsIgnoreCase agar pencarian tidak sensitif terhadap huruf besar atau kecil. Jika ditemukan program menampilkan informasi lagu tersebut, jika tidak ditemukan program menampilkan pesan bahwa lagu tidak ada dalam playlist.

Pada fungsi main, program menyediakan antarmuka menu interaktif berbasis teks dengan enam pilihan operasi menggunakan Scanner untuk membaca input pengguna. Program akan terus berjalan dalam perulangan while selama pengguna belum memilih opsi keluar yaitu pilihan nomor 6, dan setiap pilihan ditangani menggunakan struktur switch yang memanggil fungsi-fungsi yang telah didefinisikan sebelumnya.

OUTPUT :

MENAMBAHKAN LAGU :

```
=== Playlist Musik NIM: 2511533016 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: Januari
Penyanyi: Glenn Fredly
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511533016 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: 50 tahun lagi
Penyanyi: warna
Lagu berhasil ditambahkan!
```

MELIHAT PLAYLIST (MAJU) DAN (MUNDUR) :

```
=== Playlist Musik NIM: 2511533016 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
Januari - Glenn Fredly
50 tahun lagi - warna
```

```

=== Playlist Musik NIM: 2511533016 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4
50 tahun lagi - warna
Januari - Glenn Fredly

```

CARI LAGU :

```

=== Playlist Musik NIM: 2511533016 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan judul lagu: Januari
Lagu ditemukan: Januari oleh Glenn Fredly

```

HAPUS LAGU DAN LIHAT PLAYLIST :

```

=== Playlist Musik NIM: 2511533016 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Lagu pertama berhasil dihapus.

=== Playlist Musik NIM: 2511533016 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
50 tahun lagi - warna

```